

PRE-DEPLOYMENT GUIDE · END USERS

DripToken

User Operational Roadmap & Journey

1

Connect a Solana
wallet

2

Check claim eligibility

3

Submit a claim
transaction

Audience: End users (claimers) of the DripToken faucet

Purpose: Visual walkthrough of the live program experience, from discovery to repeated claims

Scope note: This is not an admin or auditor guide

1 High-Level User Journey

DISCOVERY → CONNECT → CHECK STATUS → CLAIM → REPEAT

Everything a user needs to do boils down to three repeatable actions. Limits, mode, and pause state are all enforced on-chain and surfaced to the user through clear errors or status helpers — the client never has to guess or track this itself.

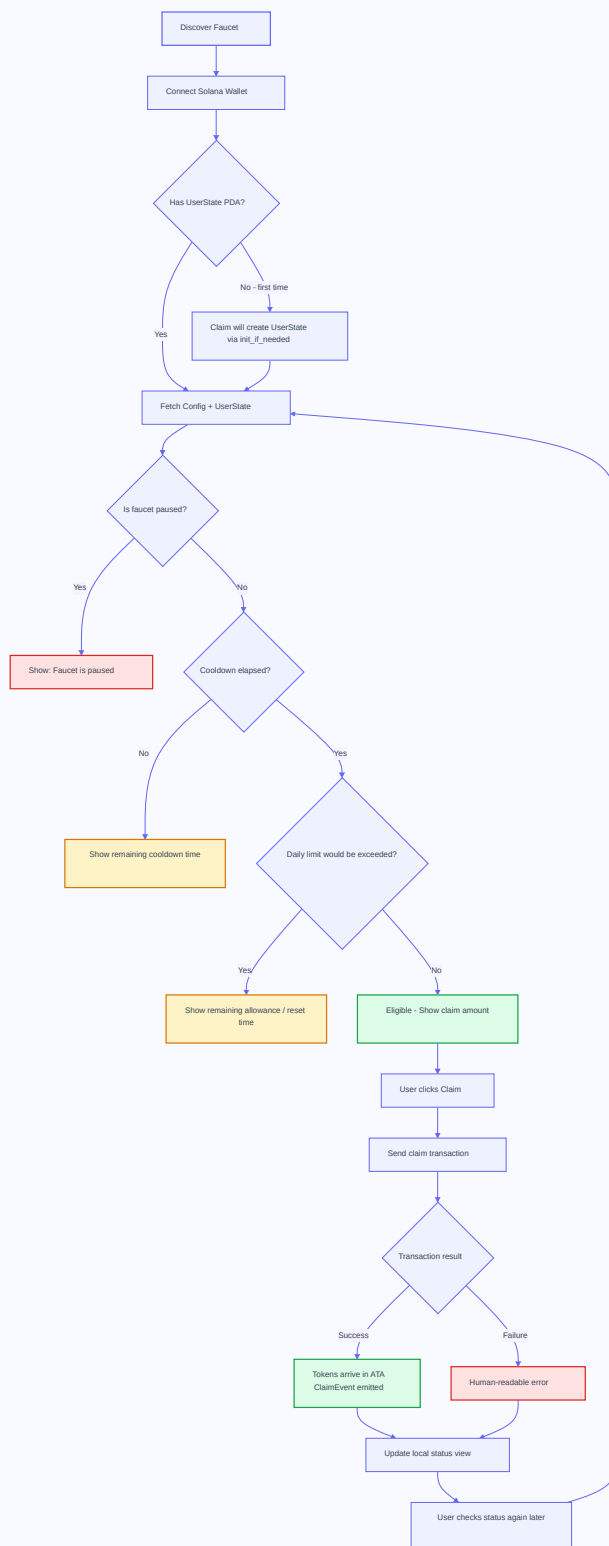


Figure 1 — End-to-end decision flow from wallet connection through claim resolution

2 Claim Sequence

WHAT ACTUALLY HAPPENS BETWEEN USER, WALLET, CLIENT, AND PROGRAM

Behind the single "Claim" button, the client reads on-chain config and user state, computes eligibility locally, then submits one instruction. The program re-checks every rule on-chain before it moves any tokens, and it updates **UserState** **before** the token transfer to prevent double-claims.

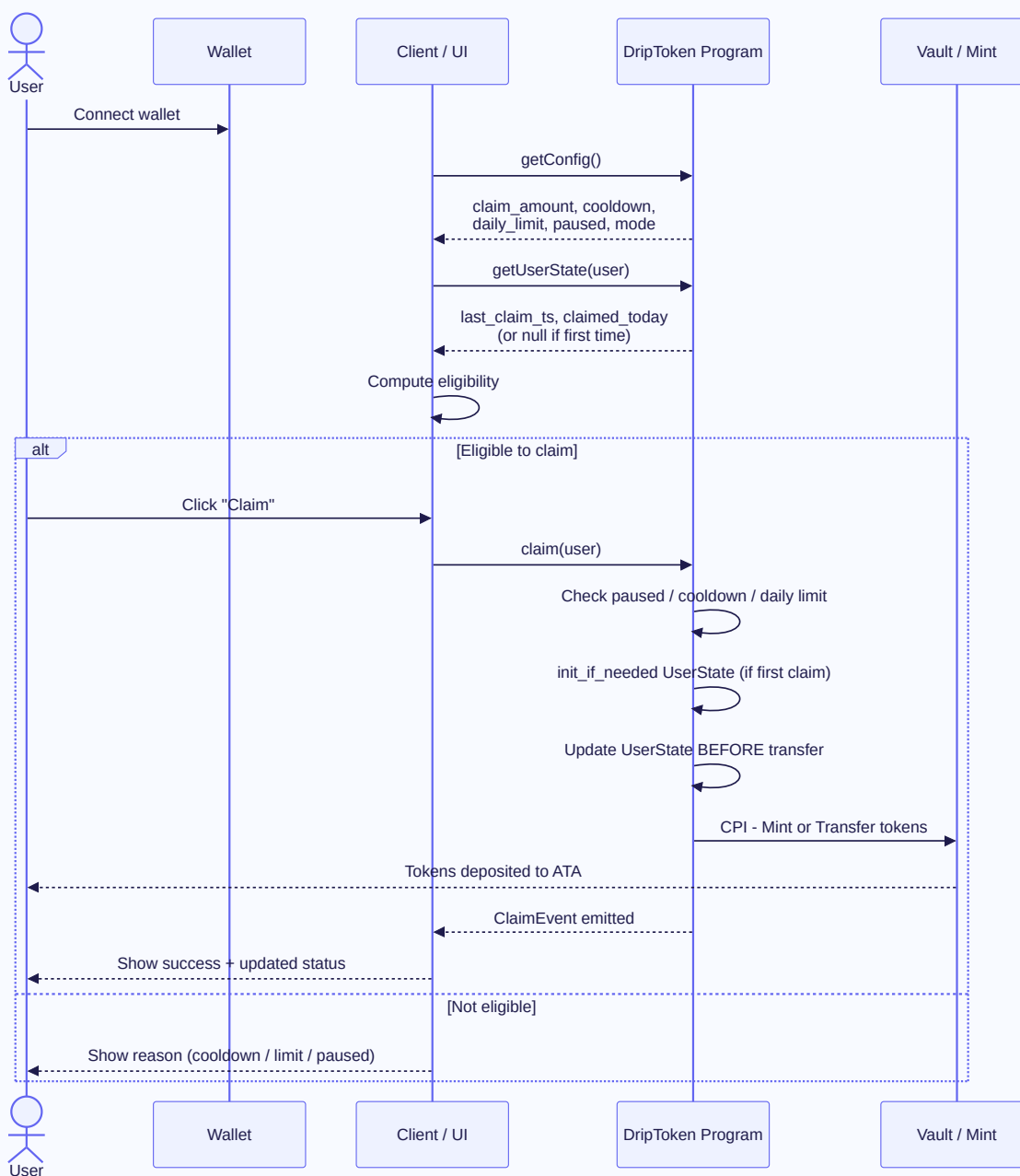


Figure 2 — Interaction sequence for a single claim attempt

3 Eligibility States

THE STATES A USER'S CLAIM STATUS CAN BE IN AT ANY MOMENT

A user is always in exactly one of these states. Three of them are simply "come back later" states — paused, on cooldown, or daily-limit reached — and each resolves back into a fresh status check automatically.

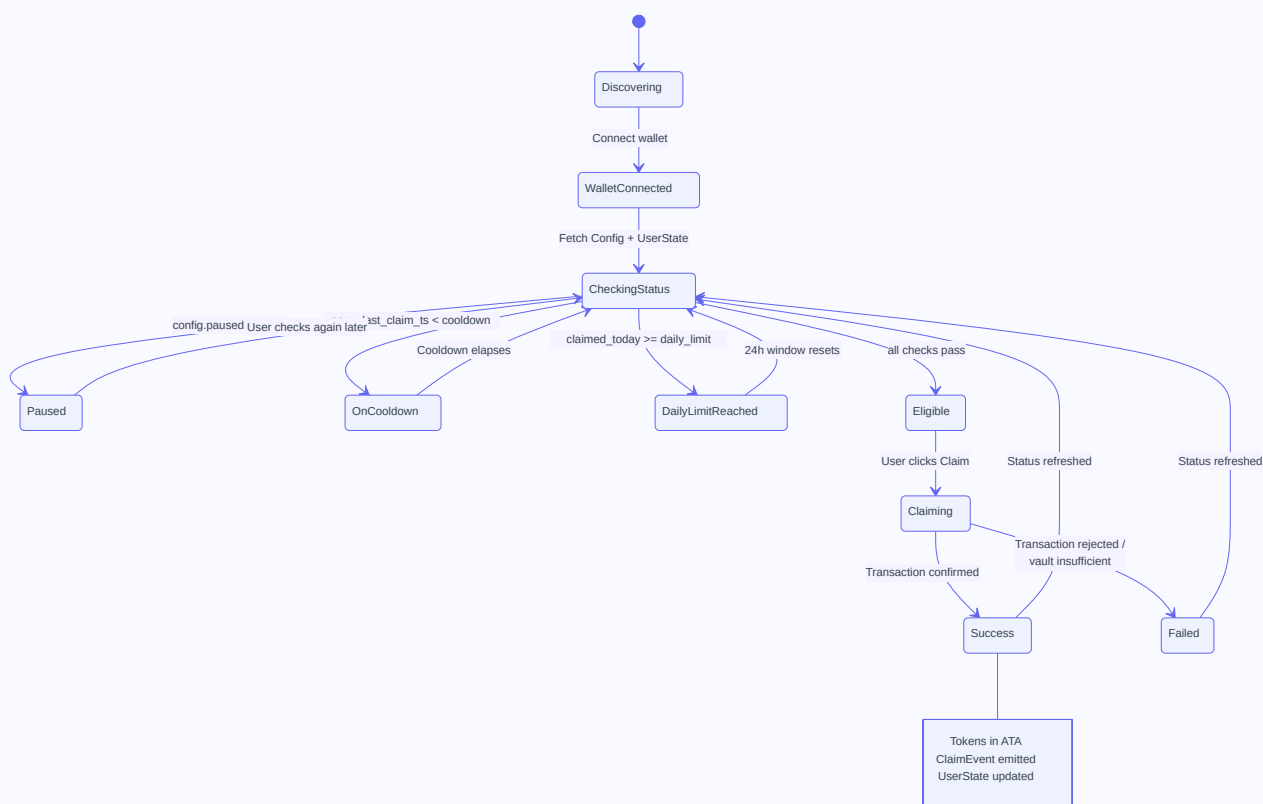


Figure 3 — Status lifecycle, from first check through claim resolution

Paused

Admin has paused the program. No claims possible until unpaused.

On Cooldown

`now - last_claim_ts < cooldown_seconds`. Wait it out.

Daily Limit Reached

`claimed_today ≥ daily_limit`. Resets on the 24h window.

4 Handling Outcomes

WHAT THE USER SEES FOR EVERY POSSIBLE RESULT

Outcome	Typical Cause	What the User Sees / Should Do
Success	All checks passed	Tokens received; status updates
Cooldown active	Too soon after last claim	"Please wait X seconds / minutes"
Daily limit reached	Already claimed max in current 24h window	"Daily limit reached. Resets in ~Y hours"
Faucet paused	Admin paused the program	"Faucet is temporarily paused"
Insufficient vault	Transfer mode and vault is empty/low	"Faucet is currently empty – try later"
First claim	No UserState yet	Automatically initialized; claim proceeds if limits allow

4a Status Data Sources

Information shown to user	Source
Current claim amount	<code>config.claim_amount</code>
Cooldown remaining	<code>now - last_claim_ts</code> vs <code>config.cooldown_seconds</code>
Daily remaining	<code>config.daily_limit - claimed_today</code> (after day-window reset logic)
Faucet status	<code>config.paused</code>
Mode (informational)	Mint or Transfer

5 What the User Never Has to Worry About

- ✓ Creating the Config PDA (admin only)
- ✓ Mint authority or vault funding (admin / operator responsibility)
- ✓ Choosing claim amount (fixed by config)
- ✓ Manually creating the UserState account — `init_if_needed` handles it
- ✓ Distinguishing Mint vs Transfer mode for the claim action — the program performs the correct CPI

6 Minimal Client Interaction Surface

All limit and pause logic lives on-chain — the client only reads state and submits the transaction:

```
const config = await getConfig(connection, programId);
const userState = await getUserState(connection, programId, wallet.publicKey);

// Display eligibility based on config + userState + current Clock

if (eligible) {
  await claim(provider, program, wallet.publicKey);
  // refresh status
}
```

7 User Mental Model

THE WHOLE PROGRAM, BOILED DOWN TO ONE LOOP

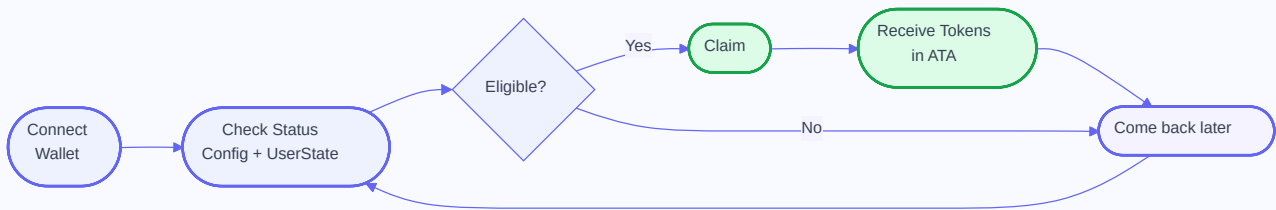


Figure 4 — The repeatable loop every returning user follows

"Connect once, check status, claim if eligible, come back later — repeat."

The program is intentionally simple: one claim action, clear limits, and automatic account creation for first-time users. Day boundaries reset automatically on-chain via a 24h rolling window based on `last_day_ts` — no registration or extra accounts required beyond a normal Solana wallet and ATA.